



CSC 2032 — Object Oriented Programming

(Group Project)

Fashion Design & Clothing Management System



Submitted By :-

- AS20240586 - K.Thoshika Chamodi
- AS20240448 – M.M.Savidya Kaumadi

Table of Content

1.Introduction

- 1.1 Introduction
- 1.2 Background
- 1.3 Problem Statement
- 1.4 Objective
- 1.5 Scope

2. System Overview

- 2.1 Proposed Solution
- 2.2 User Roles
- 2.3 Use Case Diagram
- 2.4 Functional Requirements
- 2.5 Non-Functional Requirements

3.Object-Oriented Design

- 3.1: Class Responsibilities
- 3.2 Applying Core OOP Principles
 - 3.2.1 Encapsulation
 - 3.2.2 Inheritance
 - 3.2.3 Polymorphism
 - 3.2.4 Abstraction
- 3.3 Enumeration
- 3.4 UML Class Diagram

4. Database Design

- 4.1 Entity-Relationship Diagram

5. System Architecture

5.1 Suggested Package Structure

6. Git Collaboration Workflow

7. Testing Plan

8. Future Enhancements

9. Conclusion

1.INTRODUCTION

1.1 Introduction

The Fashion Design and Clothing Management System is a comprehensive web-based application developed to improve the efficiency of fashion business operations through digital automation. The system integrates customer management, designer services, product management, inventory control, order processing, and payment handling into a single platform. It provides an easy-to-use interface for customers to browse clothing collections, place customized orders, and make secure payments while allowing designers to upload and manage their fashion creations. Administrators are provided with advanced tools to monitor user activities, manage products, supervise inventory, and generate business reports. The system has been developed using Java Object-Oriented Programming principles together with HTML, CSS, JavaScript, and MySQL to ensure scalability, security, and maintainability.

1.2 Background

The fashion industry is growing rapidly with the increasing use of online shopping and digital services. Customers prefer buying clothes online because it is easy, fast, and convenient. However, many fashion businesses still use manual methods to manage customer information, products, inventory, and orders. These methods can cause errors, delays, and poor record management. The Fashion Design and Clothing Management System was developed to automate these activities and provide a single platform where customers, designers, and administrators can manage fashion-related services efficiently.

1.3 Problem Statement

Many fashion businesses still use manual methods to manage customer information, inventory, and orders, leading to errors, delays, and inefficient operations. These traditional systems make it difficult to maintain accurate records and provide timely customer service. To overcome these challenges, the Fashion Design and Clothing Management System was developed to automate business processes, improve data management, and enhance communication between customers, designers, and administrators.

1.4 Objectives

- To develop a secure and user-friendly web-based Fashion Design and Clothing Management System.
- To simplify customer registration, product browsing, order management, and online payment processing.
- To enable designers to upload, update, and manage fashion designs efficiently.
- To provide administrators with tools to manage users, products, inventory, and orders.
- To improve business efficiency by reducing manual work and ensuring accurate data management.
- To develop a scalable and maintainable system using Object-Oriented Programming principles.

1.5 Scope

The Fashion Design and Clothing Management System is designed to manage the main activities of a fashion business. Customers can register, log in, browse products, place orders and make payments. Designers can upload and manage clothing designs, while administrators can manage users, products, inventory, orders, and payments. The system stores all information securely in a database and helps improve the efficiency of daily business operations. Advanced features such as AI recommendations, mobile applications, and virtual fitting rooms are not included in the current system and can be added in future versions.

2. System Overview

2.1 Proposed Solution

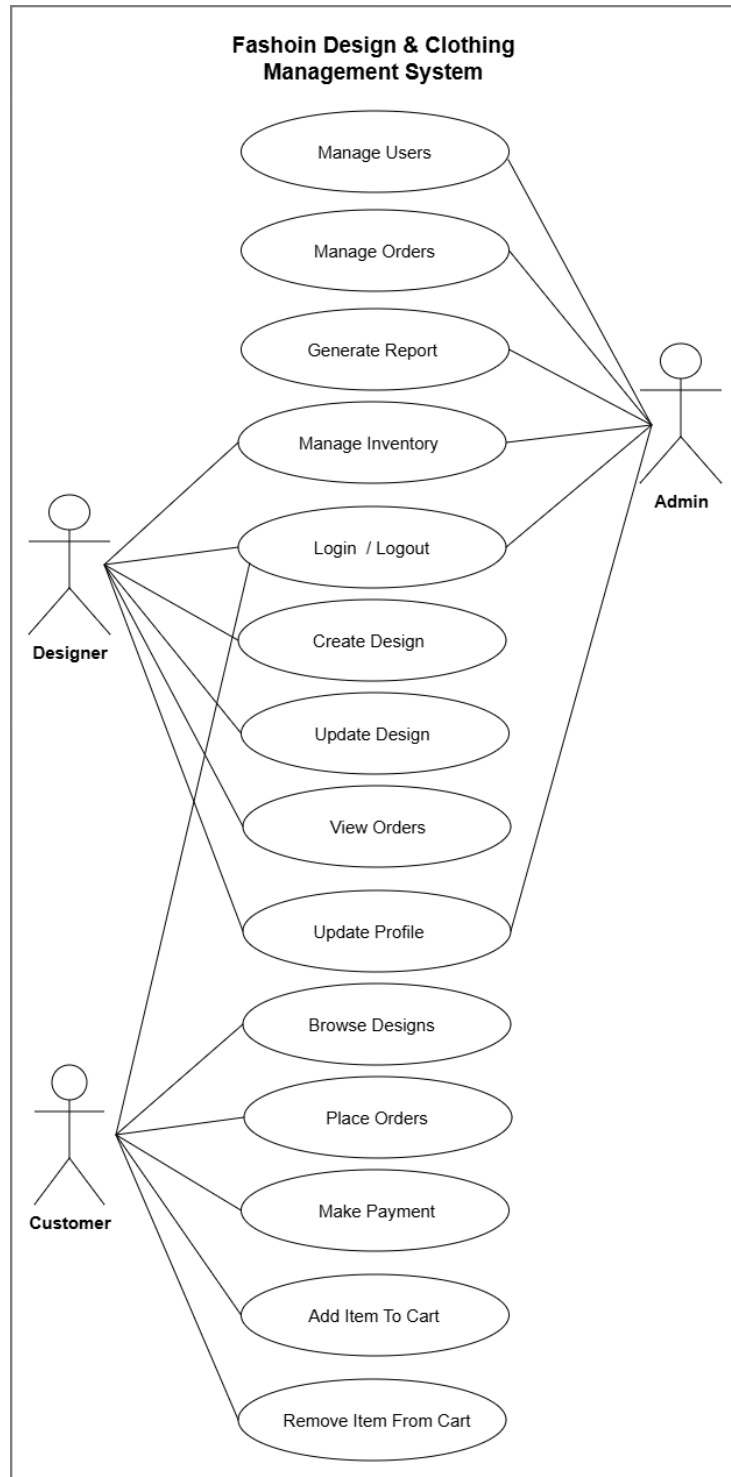
The proposed Fashion Design and Clothing Management System is a web-based application developed using HTML, CSS, JavaScript, Java (Servlets/JSP), and MySQL. The system automates customer registration, product management, order processing, inventory management, payment handling, and designer management. It provides separate dashboards for Customers, Designers, and Administrators, ensuring secure access and efficient management of fashion business operations.

2.2 User Roles

User Role	Description	Key Responsibilities
Customer	Uses the system to purchase and customize clothing products.	Register and log in, browse products, add items to the cart, place orders, make payments, and track order status.
Designer	Manages fashion designs and product information.	Upload and update clothing designs, manage product details, respond to customization requests, and monitor product orders.
Admin	Oversees the overall operation of the system.	Manage users, products, inventory, orders, payments, generate reports, and maintain system security.

2.3 Use Case Diagram

Below the figure shows the main interactions between the three user roles and the system.



2.4 Functional Requirements

- **User Registration** : The system allows customers and designers to create a new account.
- **User Login** : The system allows users to log in securely using their username and password.
- **Product Management** : The system allows designers and administrators to add, update, delete, and manage clothing products.
- **Order Management** : The system allows customers to place, view, and manage their orders.
- **Shopping Cart** : The system allows customers to add, update, and remove products from the shopping cart.
- **Inventory Management** : The system could update product stock after each order.
- **Designer Management** : The system allows designers to upload and manage their clothing designs.
- **User Management** : The system allows administrators to manage customer and designer accounts.
- **View Report** : The system allows administrators to view reports on sales, orders, and inventory.

2.5 Non-Functional Requirements

- **Performance** : The system should respond quickly and support multiple users at the same time.
- **Security** : The system should protect user information and provide secure login.
- **Reliability** : The system should work correctly with minimum errors and downtime.
- **Usability** : The system should have a simple and easy-to-use interface.
- **Scalability** : The system should support future growth and additional features.
- **Maintainability** : The system should be easy to update and maintain.
- **Availability** : The system should be available whenever users need it.
- **Data Integrity** : The system should store accurate and consistent data.

3. Object-Oriented Design

The Fashion Design and Clothing Management System was developed using Object-Oriented Programming (OOP). The system is divided into different classes such as User, Customer, Designer, Admin, Design, Order, Payment, Cart, and Inventory. Each class has its own responsibility and works together to perform the system functions. Using OOP makes the system easy to understand, maintain, and improve in the future.

3.1: Class Responsibilities

Class	Responsibility
User (Abstract)	Serves as the parent class for all system users and stores common user information such as user ID, name, email, password, phone number, and address.
Customer	Allows customers to place orders, view order history, make payments, and update their profile information.
Designer	Creates and updates fashion designs, views customer design orders, and manages clothing inventory.
Admin	Manages users, customer orders, inventory, and generates system reports.
Design	Stores clothing design details such as design name, fabric, price, category, pattern, sketch image, description, and status.
Fabric	Stores information about available fabrics including fabric name, color, and price per unit.
Cart	Represents the customer's shopping cart and manages the products selected before placing an order.
Order	Stores customer order information, calculates the total amount, and manages order placement and cancellation.
OrderItem	Represents individual products within an order and calculates the total price for each ordered item.
CartItem	Stores individual items added to the shopping cart, including design, size, quantity, and custom notes.
Payment	Handles payment processing, stores payment information, and generates payment receipts.
Delivery	Manages delivery information including delivery address, delivery date, and delivery status.
Inventory	Maintains stock information, updates product quantities, and manages inventory records.
PaymentMethod (Enumeration)	Defines the available payment methods such as Cash and Online.
Size (Enumeration)	Defines the available clothing sizes such as XS, S, M, L, XL, XXL, and NONE.

3.2 Applying Core OOP Principles

3.2.1 Encapsulation

Encapsulation is used to protect the data stored within each class. In this system, class variables are declared as private, and they are accessed through public getter and setter methods. This prevents unauthorized access and ensures that data can only be modified through controlled methods. For example, customer information, payment details, and product information are securely managed using encapsulation.

A screenshot of a code editor with a dark background. At the top, there are tabs for 'na.sql', 'Admin.java', 'Designer.java', 'Fabric.java', 'Inventory.java', and 'Order.java'. The 'Admin.java' tab is active. The code is as follows:

```
1 package com.fashiondesign.model;
2
3 public class Payment { 2 usages
4     private String paymentId; 2 usages
5     private String orderId; 3 usages
6     private PaymentMethod method; 3 usages
7     private double amount; 3 usages
8     private String status; // PENDING, COMPLETED, FAILED 4 usages
9     private String paymentDate; 2 usages
10
11     public Payment() { no usages
```

3.2.2 Inheritance

Inheritance is used to reduce code duplication by allowing one class to inherit the properties and methods of another class. In this system, the Customer, Designer, and Admin classes inherit common attributes such as user ID, name, email, username, and password from the User class. This allows all user types to share common functionality while adding their own specific features. User is superclass and Admin, Customer and Designer are subclasses of User superclass.

```
Admin.java  Cart.java  User.java x  UserDAO.java

1  package com.fashiondesign.model;
2
3
4  @ public abstract class User { 3 usages 3 inheritors
5      private String userId; 3 usages
6      private String name; 3 usages
7      private String email; 4 usages
8      private String password; 4 usages
9      private String phone; 3 usages
10     private String address; 3 usages
11
12     public User() { 3 usages
13     }
14
15     public User(String userId, String name, String email, String password, String phone, String address) { 3 usages
16         this.userId = userId;
17         this.name = name;
18         this.email = email;
19         this.password = password;
20         this.phone = phone;
21         this.address = address;
22     }
23
```

```
ema.sql  Admin.java  Designer.java x  Cart.java  CartItem.java  Customer.java  Delivery.java  D

1  package com.fashiondesign.model;
2
3  import java.util.ArrayList;
4  import java.util.List;
5
6
7  public class Designer extends User { no usages
8  private String designId; 4 usages
9  private String speciality; 3 usages
10 private List<Design> designs = new ArrayList<>(); 4 usages
11
12     public Designer() { 1 usage
13     }
14
15     public Designer(String userId, String name, String email, String password, String phone, 1 usage
16                     String address, String designId, String speciality) {
17         super(userId, name, email, password, phone, address);
18         this.designId = designId;
19         this.speciality = speciality;
20     }
21
22     public Design createDesign(String designName, String category, String pattern, no usages
23                               String fabricId, double price, String sketchImage, String description) {
```

```
schema.sql  Admin.java x
1 package com.fashiondesign.model;
2
3
4 public class Admin extends User { no usages
5     private String adminId; 3 usages
6
7     public Admin() { 2 usages
8     }
9
10    public Admin(String userId, String name, String email, String password, String phone,
11                  String address, String adminId) {
12        super(userId, name, email, password, phone, address);
13        this.adminId = adminId;
14    }
15
16    public void manageUsers() { no usages
17        // Delegates to UserDAO
18    }
19
20    public void manageOrders() { no usages
21        // Delegates to OrderDAO
22    }
23
```

```
schema.sql  Admin.java  Cart.java  CartItem.java  Customer.java x
1 package com.fashiondesign.model;
2
3
4 public class Customer extends User { no usages
5     private String customerId; 4 usages
6
7     public Customer() { no usages
8     }
9
10    public Customer(String userId, String name, String email, String password, String phone,
11                    String address, String customerId) {
12        super(userId, name, email, password, phone, address);
13        this.customerId = customerId;
14    }
15
16    @ public Order placeOrders(Order order) { no usages
17        order.setCustomerId(this.customerId);
18        return order;
19    }
20
21    public void viewOrders() { no usages
22        // Actual retrieval is done through OrderDAO.
23    }
```

3.2.3 Polymorphism

Polymorphism allows different classes to perform similar operations in different ways. In this system, methods such as `getRole()` method can behave differently depending on whether the logged-in user is a Customer, Designer, or Administrator. This improves flexibility and allows the same method name to perform role-specific tasks.

```
public class Designer extends User { no usages
    @Override no usages
    public String getRole() {
        return "DESIGNER";
    }

    public String getDesignId() { no usages
        return designId;
    }

    public void setDesignId(String designId) { no usages
        this.designId = designId;
    }

    public String getSpeciality() { no usages
        return speciality;
    }

    public void setSpeciality(String speciality) { no usages
        this.speciality = speciality;
    }
}
```

3.2.4 Abstraction

Abstraction is used to hide complex implementation details from users while displaying only the necessary functionality. In this system, users interact with simple interfaces such as registration forms, product pages, shopping carts, and dashboards without knowing how the internal business logic or database operations are performed. This makes the system easier to use while keeping the code organized and secure. The user class of this system is abstract class.

```
Admin.java  Cart.java  User.java x  UserDAO.java

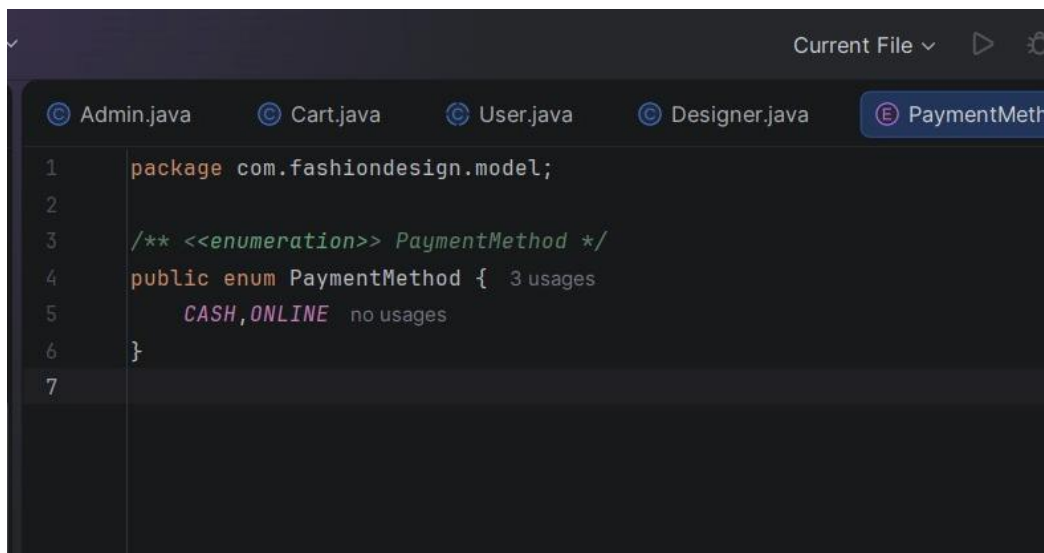
1  package com.fashiondesign.model;
2
3
4  @ public abstract class User { 3 usages 3 inheritors
5      private String userId; 3 usages
6      private String name; 3 usages
7      private String email; 4 usages
8      private String password; 4 usages
9      private String phone; 3 usages
10     private String address; 3 usages
11
12     public User() { 3 usages
13     }
14
15     public User(String userId, String name, String email, String password, String phone, String address) { 3 usages
16         this.userId = userId;
17         this.name = name;
18         this.email = email;
19         this.password = password;
20         this.phone = phone;
21         this.address = address;
22     }
23
```

3.3 Enumeration

An enumeration (enum) is a special Java data type used to define a fixed set of constant values. It ensures that only predefined values can be assigned to a variable, reducing errors and improving code readability. In the Fashion Design and Clothing Management System, enumerations are used for attributes that have a limited number of valid options. For example, the Size enum contains values such as XS, S, M, L, XL, XXL, and NONE, while the PaymentMethod enum contains CASH and ONLINE. When a customer places an order, they can only choose one of these predefined payment methods, ensuring that invalid values cannot be entered into the system. This makes the system more reliable, easier to maintain, and consistent.



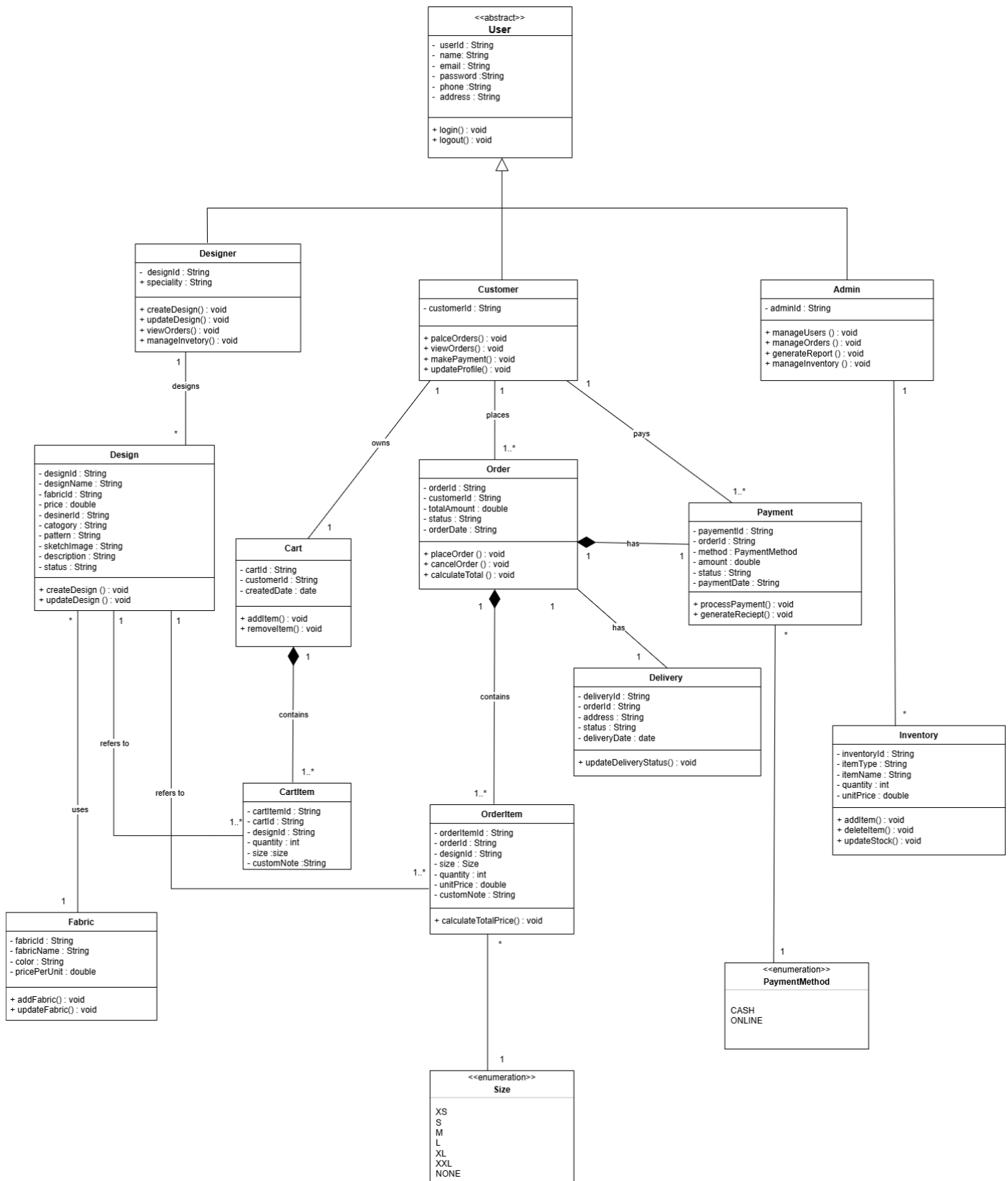
```
a    © Fabric.java    © Inventory.java    © Order.java
1    package com.fashiondesign.model;
2
3    /** <<enumeration>> Size */
4    public enum Size { 6 usages
5        XS, S, M, L, XL, XXL, NONE no usages
6    }
7
```



```
Current File v
© Admin.java    © Cart.java    © User.java    © Designer.java    © PaymentMeth
1    package com.fashiondesign.model;
2
3    /** <<enumeration>> PaymentMethod */
4    public enum PaymentMethod { 3 usages
5        CASH, ONLINE no usages
6    }
7
```

3.4 UML Class Diagram

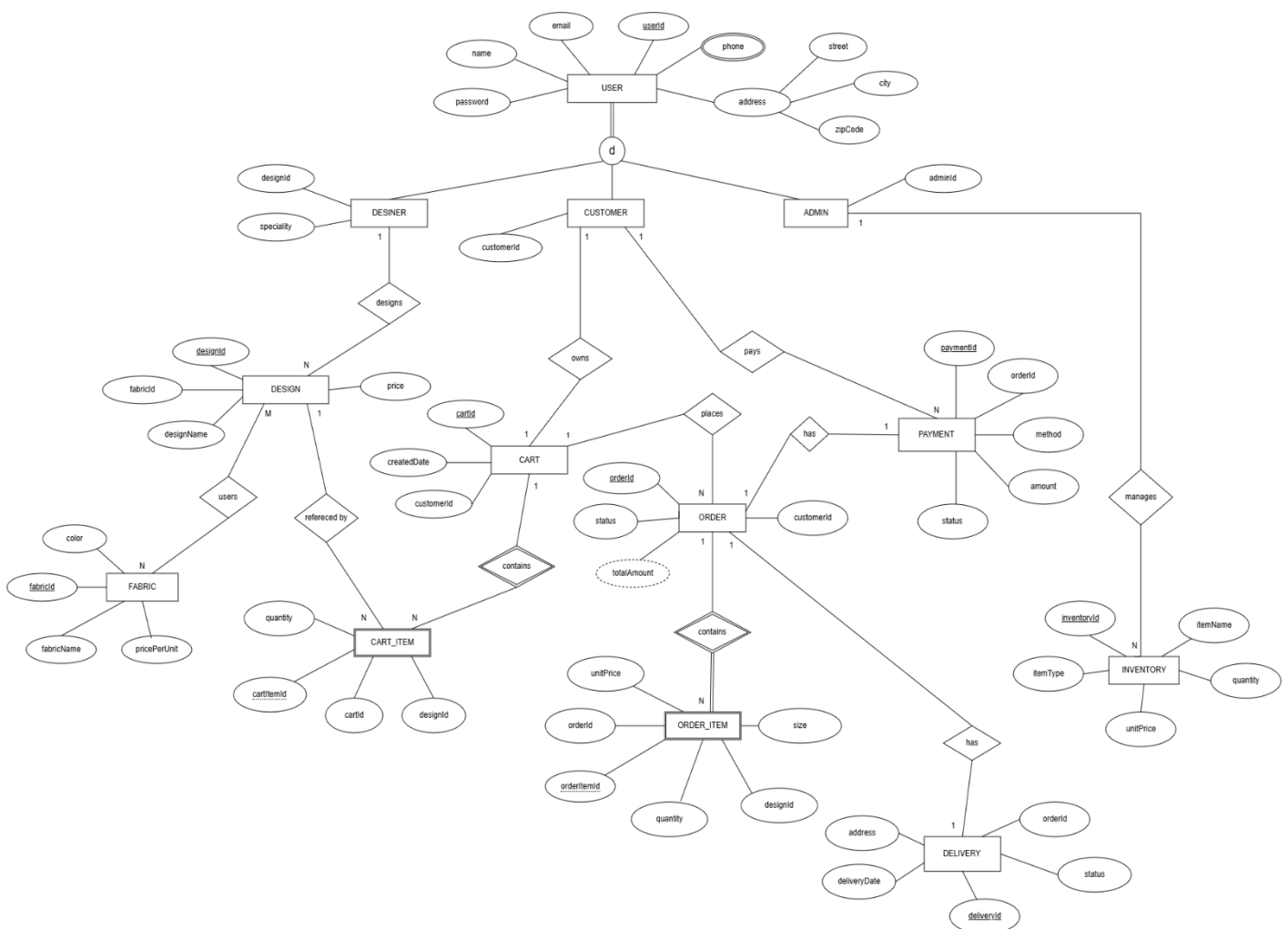
Below figure shows the main classes of the Fashion Design and Clothing Management System and how they are connected. It illustrates the relationships between classes and the use of Object-Oriented Programming concepts to organize the system.



4. Database Design

4.1 Entity-Relationship Diagram

The Entity-Relationship (ER) Diagram shows the database structure of the Fashion Design and Clothing Management System. It illustrates the main entities, their attributes, and the relationships between them, helping to organize and manage data efficiently while maintaining data accuracy and consistency.



5. System Architecture

The Fashion Design and Clothing Management System follows a three-tier architecture, which consists of the Presentation Layer, Business Logic Layer, and Database Layer.

Presentation Layer (Frontend)

- Provides the user interface.
- Developed using HTML, CSS, and JavaScript.
- Allows users to interact with the system.
- Displays forms, products, dashboards, and reports.

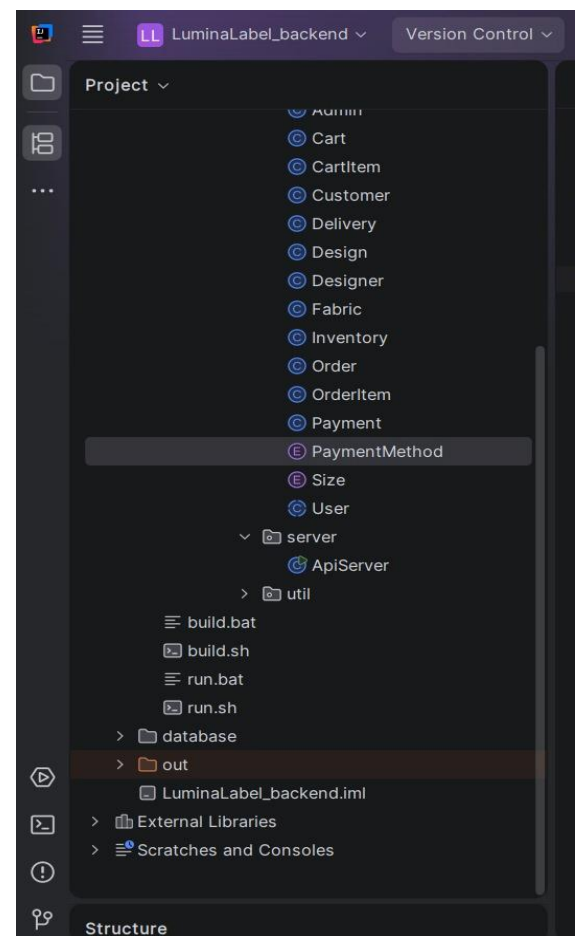
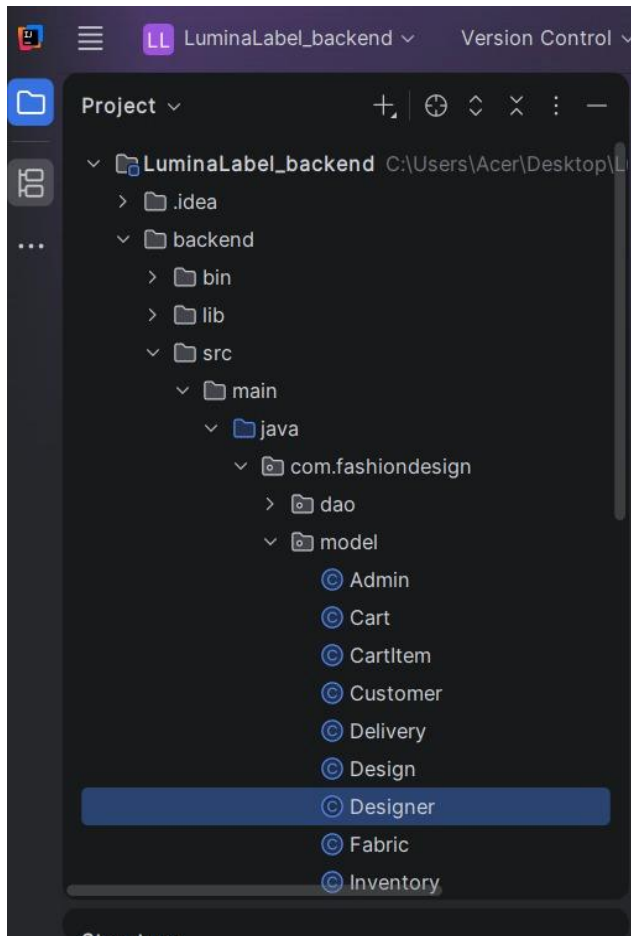
Business Logic Layer (Backend)

- Developed using Java.
- Processes user requests.
- Handles login, order management, payments, and inventory.
- Communicates between the frontend and database.

Database Layer

- Uses MySQL to store system data.
- Stores customer, designer, product, order, payment, and inventory information.
- Retrieves and updates data when requested by the application.

5.1 Suggested Package Structure



6. Git Collaboration Workflow

- **Create and Clone Repository:** Create a Git repository and clone it to each team member's local computer.
- **Develop in Branches:** Each team member works on a separate branch to develop and test their assigned features.
- **Commit and Push Changes:** Commit completed work with meaningful messages and push the changes to the remote repository.
- **Review and Merge:** Review the code using pull requests, merge approved changes into the main branch, and update the local repository with the latest version.

7. Testing Plan

Test Case	Input	Expected Result
User Registration	Valid customer details	New user account is created successfully.
User Login	Valid username and password	User is logged in and redirected to the appropriate dashboard.
Invalid Login	Incorrect username or password	Error message displayed and login is denied.
Add Product	Valid product details	Product is added successfully to the system.
Place Order	Product selected with valid quantity	Order is placed successfully and order confirmation is displayed.
Update Inventory	Product stock changes after an order	Inventory quantity is updated automatically.
Search Products	Product name or category	Matching clothing products are displayed.
Manage Fashion Design	Valid design information	Design is added or updated successfully by the designer.
View Order History	Customer account	Customer's previous orders are displayed correctly.

8. Future Enhancements

- Develop a mobile application for Android and iOS devices.
- Integrate secure online payment gateways.
- Add AI-based clothing recommendations.
- Implement a virtual fitting room feature.
- Enable real-time order tracking and notifications.
- Introduce customer reviews and product ratings.
- Support multiple languages and currencies.
- Add an analytics dashboard for business reports.

9. Conclusion

The Fashion Design and Clothing Management System is a user-friendly web application that simplifies the management of fashion products, customer orders, inventory, and payments. By applying Object-Oriented Programming concepts, the system is well-structured, secure, and easy to maintain. It improves business efficiency by reducing manual work and provides a better shopping experience for customers while helping designers and administrators manage their tasks effectively. The system also has the flexibility to be enhanced with additional features in the future.

Git Repository Link :- <https://github.com/Thoshika0586/Fashion-Design-and-Clothing-Management-System.git>

Thank You!